Task 1: Hardware-Conscious 2D Convolution

CS683 Programming Assignment 1

Dhurandhar Programming

Advanced Computer Architecture
Autumn 2026

5 September 2026

Executive summary

This report evaluates loop reordering, manual loop unrolling, cache tiling, and explicit SIMD for single-channel, zero-padded 2D convolution. All submitted kernels match the supplied naive reference within the required absolute tolerance of 10^{-3}. On the graded 2048×2048, $K = 3$ workload, the combined AVX2 implementation reached **19.95× speedup** in the final size-scaling batch. A separate complete grading-harness run reported **19.86×** and **100/100** in the repository's internal rubric.

The main result is that instruction reduction alone is insufficient when repeated whole-image passes saturate the memory hierarchy. SIMD-only speedup falls toward $2\times$ on large images even though AVX2 reduces retired instructions by about $8\times$. Cache tiling restores data reuse, allowing SIMD and register blocking to produce substantially larger end-to-end gains.

Reference result at $N = 2048$, $K = 3$

Version	Median time (ms)	Speedup
Naive reference	24.318	1.00×
Loop reorder	13.708	1.77×
Loop unroll	11.771	2.07×
Tiling	9.252	2.63×
Tiling + AVX2	2.548	9.54×
All combined	1.219	19.95×

Absolute times vary with cloud-VM scheduling and frequency, so conclusions are based mainly on within-run speedups, medians, instruction counts, and cache-counter trends.

Experimental setup

Hardware and software

Item	Configuration
Processor	Intel Xeon Platinum 8488C, 2 physical cores / 4 logical CPUs exposed
L1 data cache	48 KiB per core (96 KiB across two instances)
L2 cache	2 MiB per core (4 MiB across two instances)
L3 cache	105 MiB shared
SIMD support	SSE, AVX2/FMA, and AVX-512F
Operating system	Ubuntu 24.04, Linux 6.17 AWS kernel
Compiler	GCC 13.3.0
Submitted flags	`-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`
Profiling tool	`perf` 6.17.13

The assignment Makefile and compiler flags were not modified. Auto-vectorization remained disabled. Each timing process was pinned to logical CPU 0 with `taskset -c 0`. The supplied harness used two warm-up runs and reported the median of seven timed runs. Random input generation used seed 1234. Speedup was calculated as

$$\text{speedup} = \frac{T_{\text{naive}}}{T_{\text{optimized}}}.$$

Hardware counters were measured with `perf stat` using instructions, L1-dcache-loads, and L1-dcache-load-misses. L1-D MPKI was calculated as

$$\text{MPKI} = 1000 \times \frac{\text{L1-D load misses}}{\text{retired instructions}}.$$

SIMD instruction totals at $N = 2048$ were normalized over one warm-up and fifteen measured executions. The 128-bit and 512-bit variants were experimental analysis builds; the submitted source remains the required AVX2 implementation.

Correctness

The official harness passed its $K = 3$ and $K = 5$ smoke tests and every measured configuration. Additional checks covered odd heights, widths from 8 to 1032, kernels $K \in \{1, 3, 5, 7\}$, and several random seeds. Experimental SIMD-width variants had maximum absolute error below 2×10^{-6} against `conv_naive`.

Task 1A: loop reordering and unrolling

The naive loop order is `oy` $\rightarrow$ `ox` $\rightarrow$ `ky` $\rightarrow$ `kx`: it finishes one output pixel before moving to the next. Loop reordering changes this to `ky` $\rightarrow$ `kx` $\rightarrow$ `oy` $\rightarrow$ `ox`. A kernel weight is reused while the inner loop streams through consecutive input and output columns. The output is initialized to zero because every kernel-position pass adds only a partial sum.

The change replaces short K-element inner traversals with long unit-stride streams. This simplifies address generation, improves prefetchability, and creates an inner loop suitable for unrolling and SIMD. Its cost is that the reordered implementation makes K^2 complete passes over the output and repeatedly loads and stores partial sums.

The unrolled version advances the column index by eight and explicitly performs eight independent scalar updates. It reduces loop-control branches and exposes instruction-level parallelism. An eight-way factor is safe because the supplied interface guarantees that W is a multiple of eight. At $N = 2048$, $K = 3$, unrolling improves speedup from 1.77$\times$ to 2.07$\times$, but it does not remove repeated output traffic.

At $N = 4096$, the 64 MiB output makes that repeated-pass cost dominant: reorder and unroll fall below the naive baseline, while cache-aware and combined implementations retain speedup.

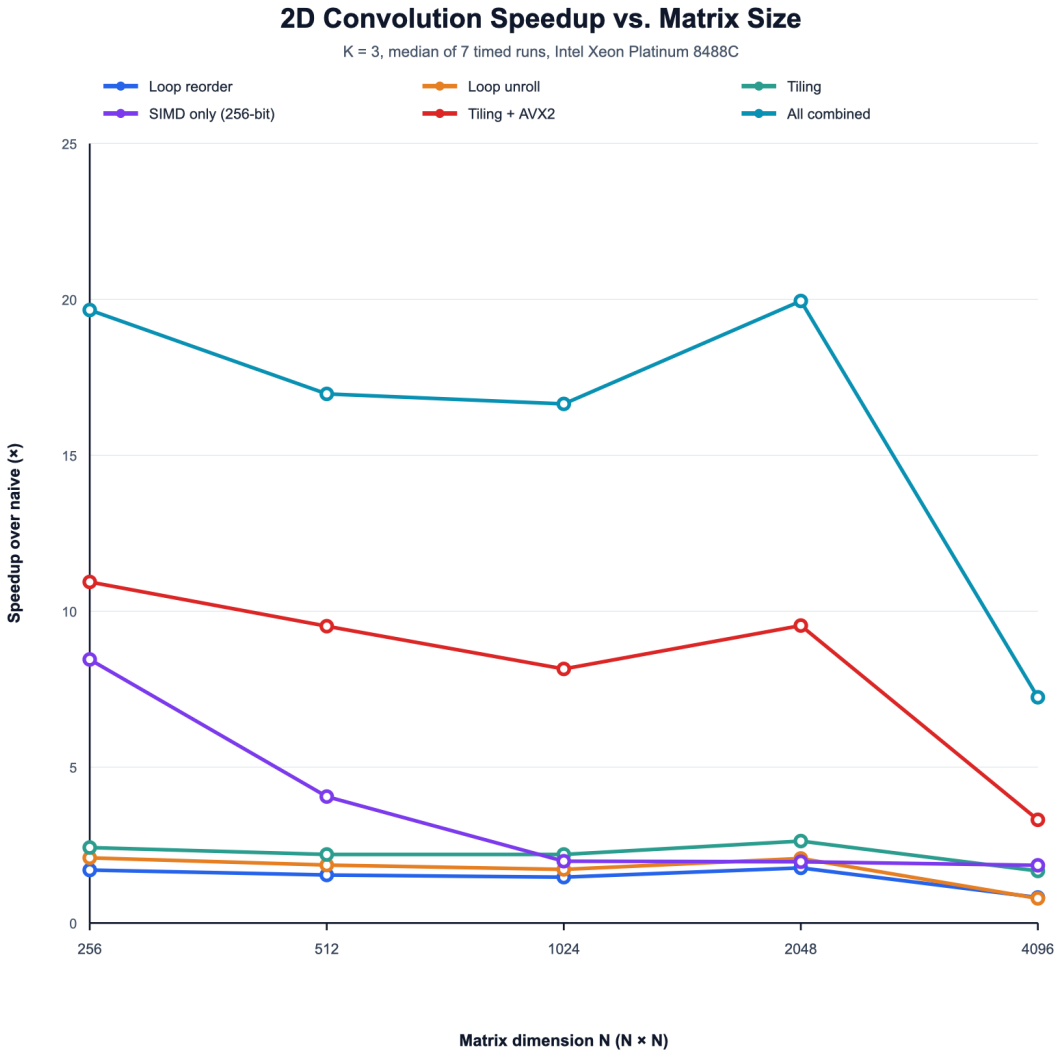


Figure 1: Speedup of the individual and combined optimization stages across matrix sizes.

Task 1B: cache tiling

The tiled kernel divides the output into horizontal strips, initializes one strip, and applies all K^2 kernel-position passes before advancing. This reduces the reuse distance for partial output values. Tile heights of 1, 2, 4, 8, 16, 32, and 64 rows were evaluated; representative results are shown below.

For $K = 3$, an approximation to the strip working set is

$$4W\,[T + (T + 2)] \text{ bytes},$$

where T is tile height. At $W = 2048$, a one-row tile uses approximately 32 KiB for one output row and three input rows, fitting in the 48 KiB L1-D cache. A two-row tile is close to L1 capacity, while four rows require roughly 80 KiB. This predicts the measured MPKI jump at $N = 2048$: from 0.818 for one row to 6.631 for four rows.

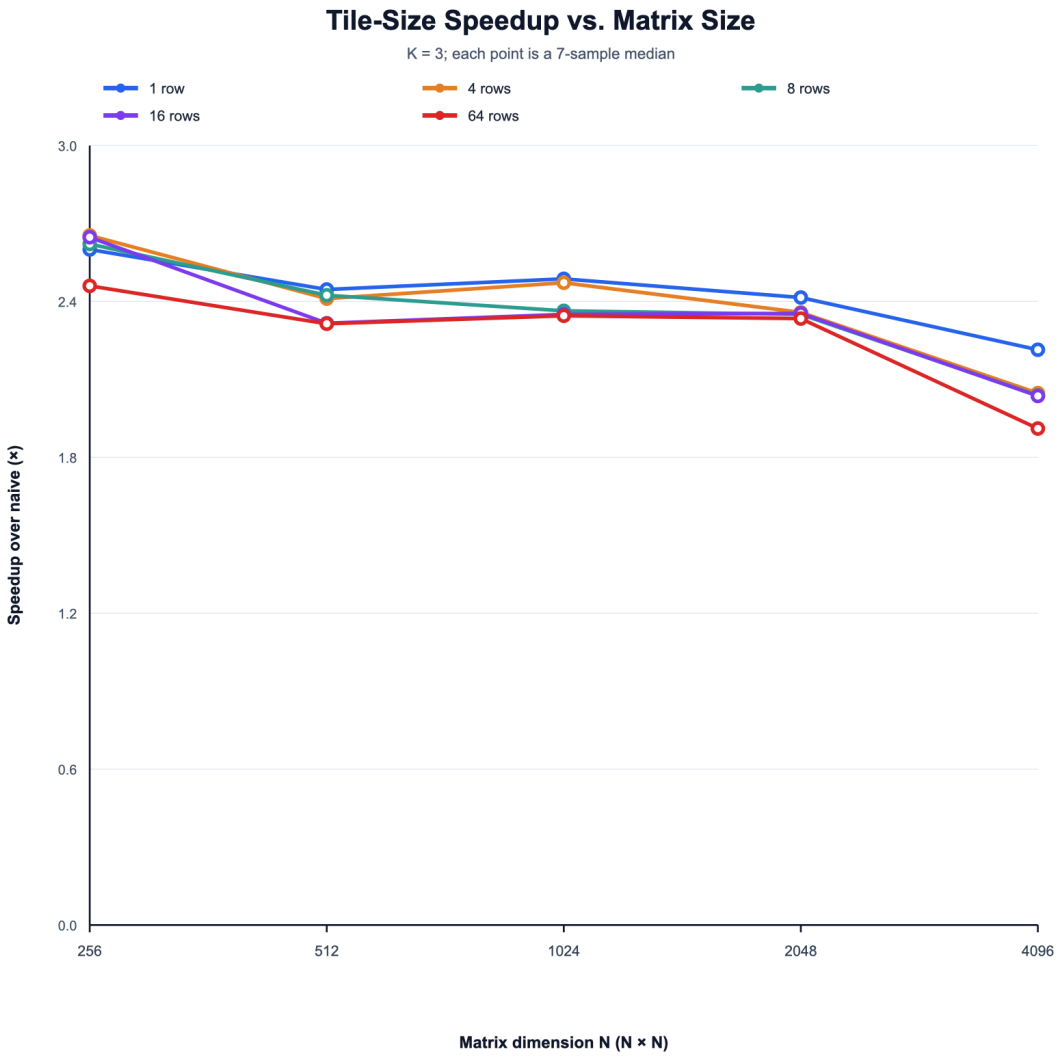


Figure 2: Speedup obtained with different horizontal-strip heights.

L1-D cache behavior

N	Naive	1 row	4 rows	8 rows	16 rows	64 rows
256	0.046	0.518	0.573	0.535	0.554	7.459
512	0.033	0.522	0.571	0.629	7.531	7.545
1024	0.037	0.612	0.780	7.395	5.397	5.358
2048	0.060	0.818	6.631	7.082	6.961	7.058
4096	0.326	1.392	5.860	5.749	5.811	6.676

The naive implementation has low MPKI because adjacent output windows overlap heavily and its partial sum stays in a register. The reordered tiled implementation retires fewer instructions but repeatedly accesses the output, so its MPKI can exceed naive. Within the tiled family, the capacity boundary is clear: MPKI rises when the strip's input-plus-output working set exceeds 48 KiB.

The one-row tile is the most robust scalar choice for medium and large images. It produces 2.42× at $N = 2048$ and 2.21× at $N = 4096$ in the controlled sweep, while the 64-row tile falls to 1.91×. The submitted scalar tiled kernel therefore uses one-row strips.

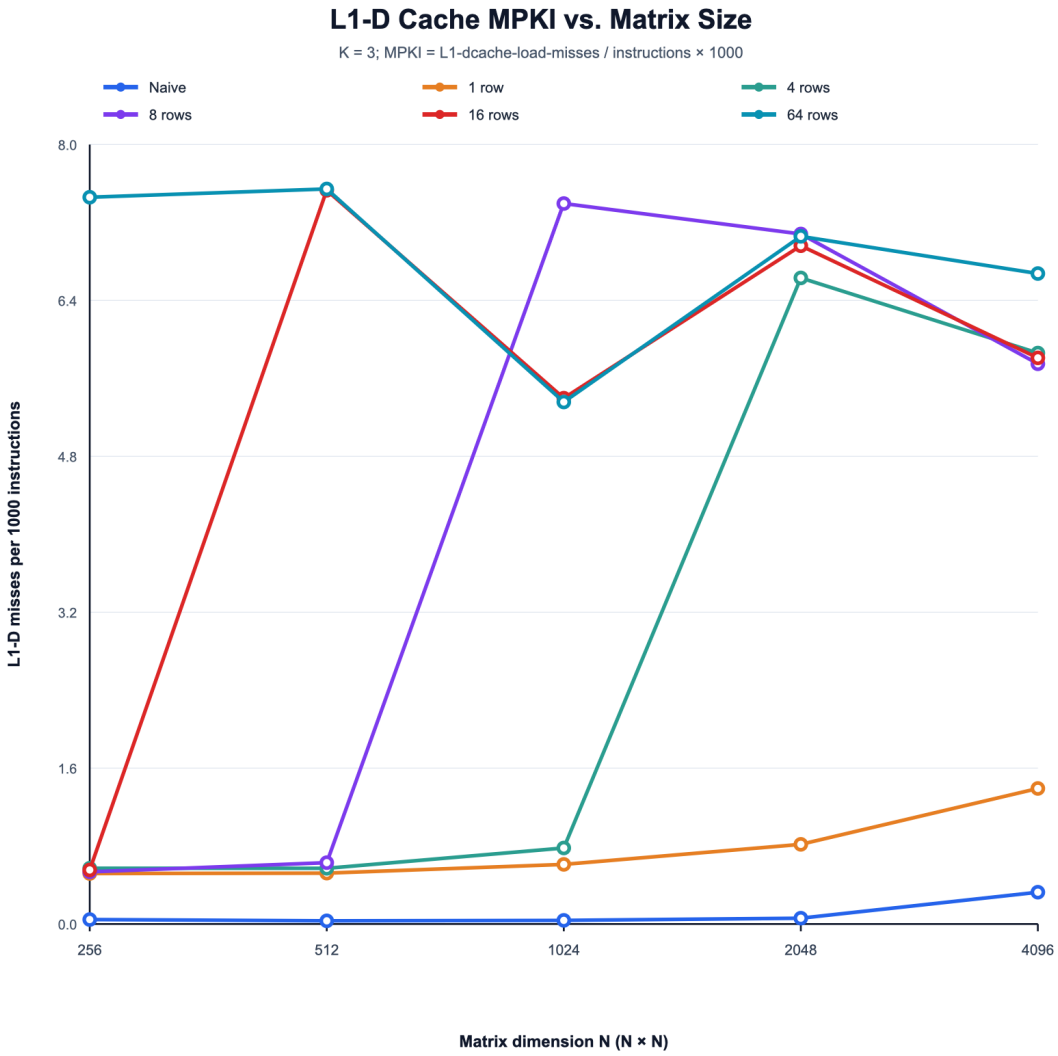


Figure 3: L1-D MPKI across matrix and tile sizes.

Task 1C: SIMD

SIMD variants broadcast a kernel weight and use unaligned vector loads because kernel offsets change input alignment. The central intrinsics were:

- 128-bit: `_mm_set1_ps`, `_mm_loadu_ps`, `_mm_fmadd_ps`, `_mm_storeu_ps`;
- 256-bit: `_mm256_set1_ps`, `_mm256_loadu_ps`, `_mm256_fmadd_ps`, `_mm256_storeu_ps`;
- 512-bit: `_mm512_set1_ps`, `_mm512_loadu_ps`, `_mm512_fmadd_ps`, `_mm512_storeu_ps`.

The submitted kernel uses 256-bit AVX2 because that width is enabled by the pinned Makefile. SSE and AVX-512 versions were temporary builds used only for the required comparison.

Retired instruction count at $N = 2048$, $K = 3$

Version	Instructions per convolution	Reduction vs. naive
Naive	384.5 million	1.00×
128-bit SSE	76.3 million	5.04×
256-bit AVX2	47.9 million	8.02×
512-bit AVX-512	33.8 million	11.38×

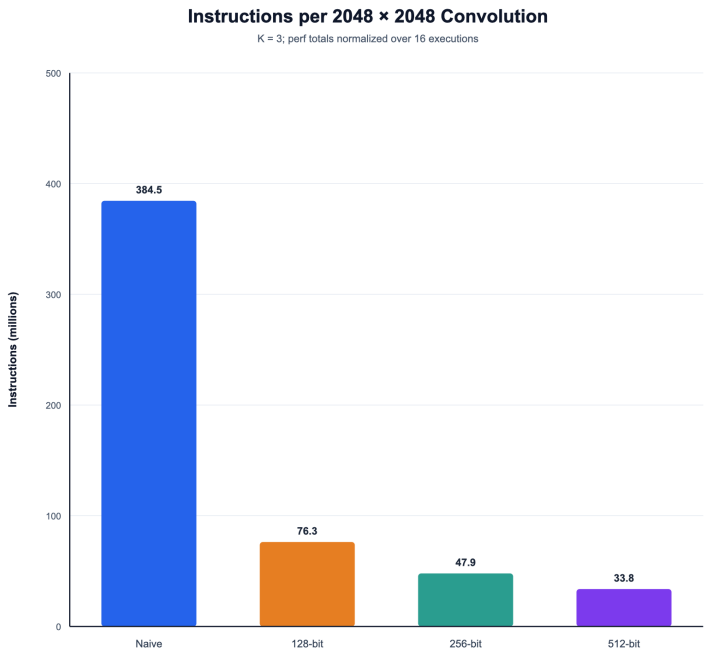


Figure 4: Retired instruction count for naive and SIMD-only versions.

SIMD width and matrix size

Figure 5 uses full-height passes to isolate SIMD from tiling. Wider registers reduce instructions strongly, but runtime speedup approaches $2\times$ for large matrices. At $N = 2048$, AVX2 retires about eight times fewer instructions than naive but reaches only $1.97\times$ speedup. The output is repeatedly read and written for each kernel position, making memory bandwidth and cache capacity the limiting factors.

AVX-512 is fastest for small matrices, but its advantage over AVX2 is negligible at $N = 4096$. Wider vectors cannot remove the memory bottleneck and may also reduce sustained core frequency on some processors. AVX2 is therefore the appropriate portable target for the submitted implementation.

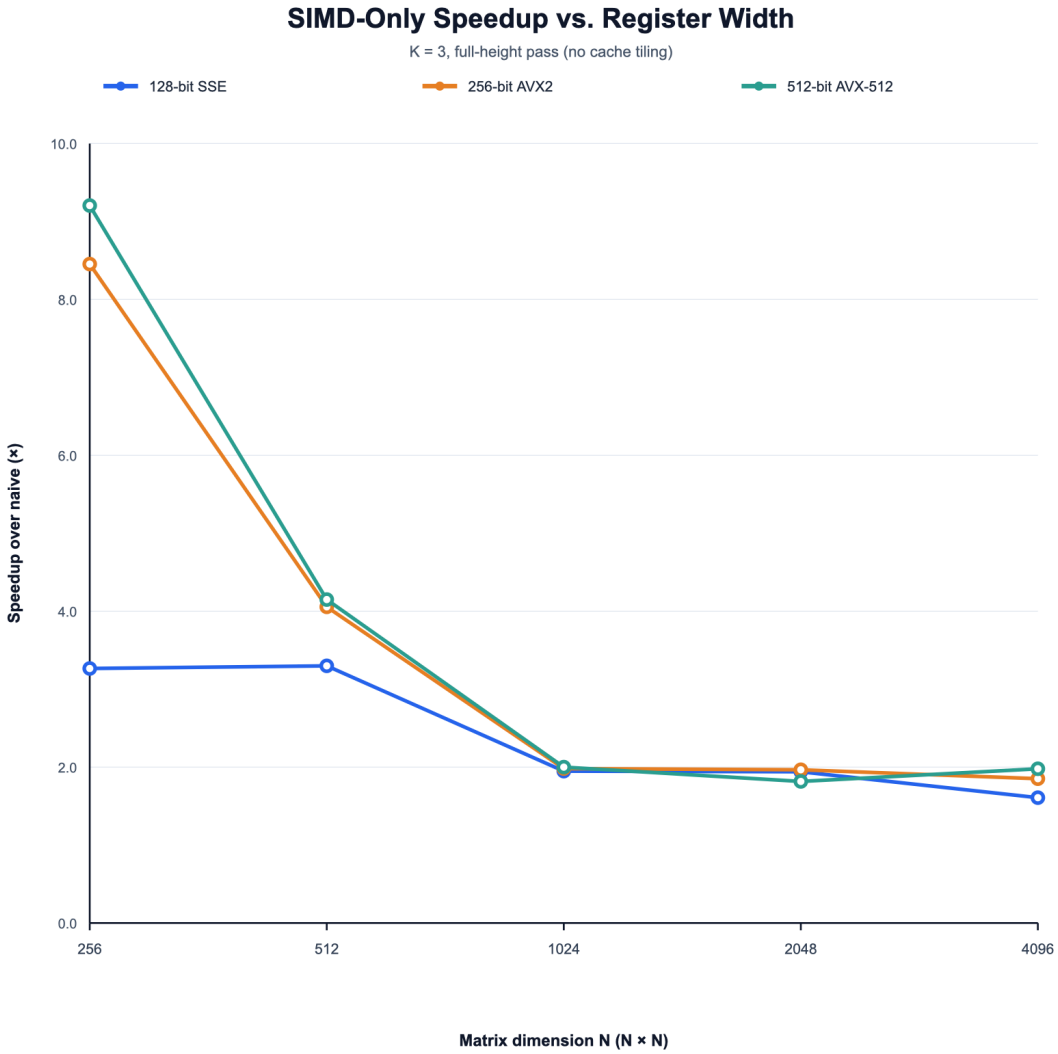


Figure 5: SIMD-only speedup for 128-, 256-, and 512-bit registers.

Task 1D: combined optimization

The submitted AVX2 stage combines a two-row cache strip with eight-float vector operations. At $N = 2048$, pure AVX2 without tiling reaches 1.97× and scalar tiling reaches 2.63×, whereas tiling plus AVX2 reaches 9.54×. Tiling removes the memory bottleneck that otherwise prevents SIMD throughput from being used effectively.

The final optimized kernel computes four AVX2 vectors, or 32 output columns, together. Four independent accumulators stay in registers while all kernel taps are processed, and each output vector is stored once. This removes the repeated partial-output traffic of the reordered stages and exposes more instruction-level parallelism. The result is 19.95× at $N = 2048$, $K = 3$.

Combined speedup decreases at $N = 4096$ because the input and output exceed the shared-cache-friendly regime and the virtualized host shows more memory and scheduling pressure. Even there, the optimized kernel remains 7.24× faster than naive, while reorder and unroll alone are slower than naive.

Kernel-size trend

For $K = 1$, setup and vector-loop overhead form a large fraction of the work, so the fully optimized version is not the fastest stage. From $K = 3$ onward, register accumulation amortizes the setup and reaches roughly 18–20×. As K grows to 7 and 9, added input traffic and a longer dependency chain slightly reduce speedup, although performance remains well above the scalar stages.

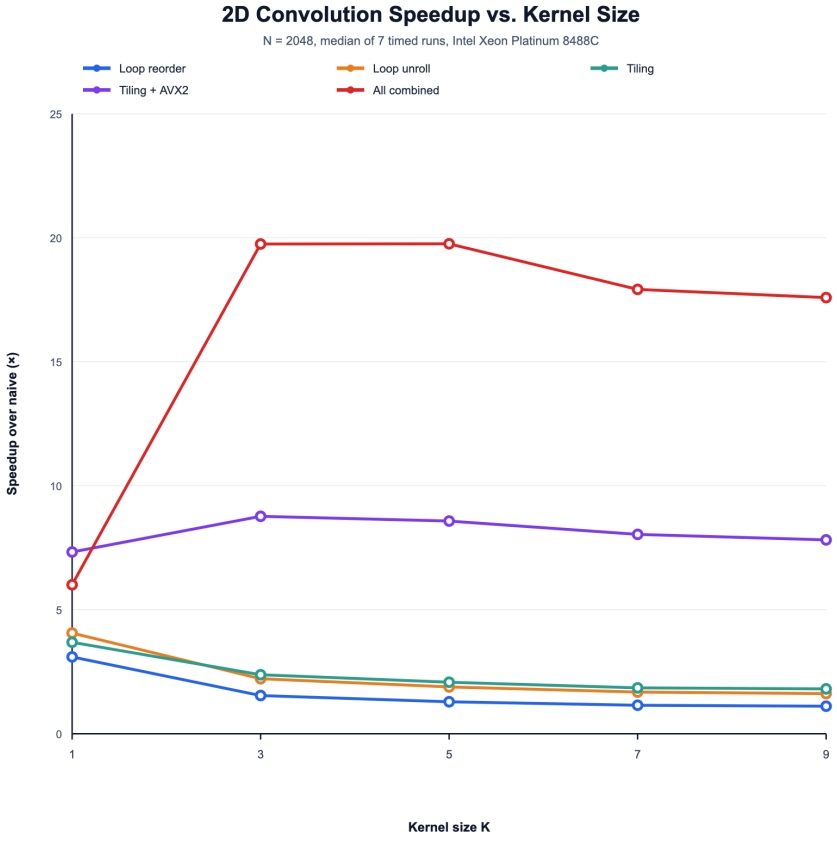


Figure 6: Speedup versus kernel size at a fixed 2048×2048 image.

Conclusions

1. Loop reordering creates regular unit-stride streams, but repeated full-output passes can become slower than naive once the image outgrows cache.
2. Eight-way scalar unrolling reduces branch overhead and exposes independent operations, producing a modest gain without reducing memory traffic.
3. Cache tiling limits reuse distance. A one-row scalar strip best matches the 48 KiB L1-D cache for large tested images; two rows perform best for the AVX2 stage.
4. SIMD greatly reduces retired instructions, but SIMD alone becomes memory-bound on large images. Instruction reduction therefore does not translate directly into equal runtime speedup.
5. Tiling and SIMD complement each other. Register-blocked AVX2 accumulation removes repeated output traffic and produces the best result: approximately 20$\times$ on the graded 2048×2048, $K = 3$ workload.

The final code modifies only the five permitted convolution source files and preserves the supplied build system and compiler flags.